

APPLIED MACHINE LEARNING
Project Report

SOCIAL NETWORKS ANALYSIS & SIMULATION

MULTI-AGENT REINFORCEMENT LEARNING FOR COOPERATION DYNAMICS
ON SOCIAL NETWORKS USING ITERATED PRISONER'S DILEMMA

Network Science × Game Theory × MARL

GitHub: [Social-Network-Simulation-Analysis](#)
Simulation Website: [Network-Simulation](#)

Table of Contents

1. Abstract
2. Introduction
 - 2.1 Background and Motivation
 - 2.2 Problem Statement
 - 2.3 Objectives
 - 2.4 Scope
 - 2.5 Research Contributions
3. Literature Review
 - 3.1 Prisoner's Dilemma and Evolutionary Game Theory
 - 3.2 Deep Q-Networks (DQN)
 - 3.3 Graph Neural Networks
 - 3.4 Network Topologies
 - 3.5 Dynamic Rewiring and Co-Evolutionary Systems
4. System Architecture
 - 4.1 Overall Design
 - 4.2 Technology Stack
 - 4.3 Co-Evolutionary Data Flow
5. Theoretical Foundations
 - 5.1 Graph Representation
 - 5.2 GCN Forward Pass
 - 5.3 Bellman Update and Training
 - 5.4 Boltzmann Exploration
 - 5.5 Prisoner's Dilemma Payoff Structure
 - 5.6 Why Graph-Based Learning Instead of Traditional DQN
6. Implementation
 - 6.1 NeuralAgent: State Vector and Personality
 - 6.2 GraphDQN Architecture
 - 6.3 Simulation Parameters and Hyperparameters
 - 6.4 Co-Evolutionary Rewiring Logic
 - 6.5 Metrics Used for Analysis
7. Experiments and Results
 - 7.1 Cooperation Dynamics Over Time
 - 7.2 Emergent Behavioural Specialisation
 - 7.3 Effect of Network Topology
 - 7.4 Impact of Co-Evolutionary Rewiring
 - 7.5 Payoff Structure vs Network Topology
 - 7.6 Phase Transition Analysis
 - 7.7 Resilience Lab: Shock and Recovery
8. Engineering Decisions and Debugging History
 - 8.1 Architectural Evolution
 - 8.2 Key Engineering Choices
 - 8.3 Comparison with Classical EGT

9. *Learnings, Shortcomings & Future Work*

9.1 Interpretation of Results

9.2 Observed Challenges and Engineering Decisions

9.3 Real-World Applications

9.4 Future Work

Conclusion

References

10. *Code and Resources*

1. Abstract

Social cooperation is fragile. Communities, organizations, and digital platforms frequently collapse from trust-driven collaboration into selfish and exploitative behaviour when individual incentives overpower collective welfare. Traditional game-theoretic models attempt to study this problem, but most assume static networks, memoryless agents, and fixed interactions — assumptions that fail to represent real social systems.

This project presents **Topology of Trust** — a **Graph Convolutional Multi-Agent Reinforcement Learning (GC-MARL)** framework designed to study adaptive cooperation dynamics on evolving social networks.

Core Components

- Graph Convolutional Deep Q-Network (**GraphDQN**) for topology-aware decision making
- **Iterated Prisoner's Dilemma** as the interaction environment
- **11-dimensional agent state representation** combining behavioural and personality features
- **Dynamic trust-based rewiring** for adaptive social restructuring

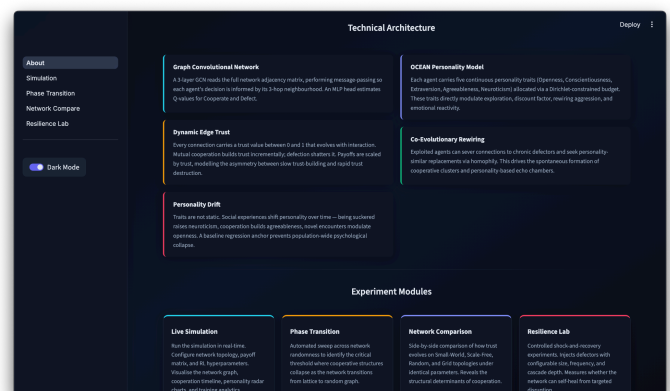
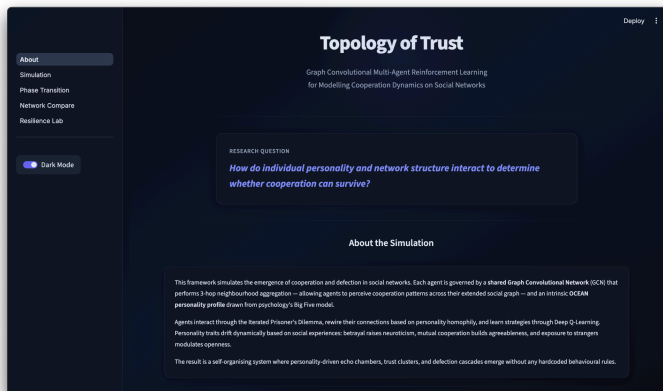
Key Experimental Observations

- Cooperation emerged naturally through graph-aware reinforcement learning
- Stable cooperation clusters and chronic defectors evolved without hard-coded behaviours
- Reward structure influenced long-term cooperation more strongly than topology alone
- Dynamic rewiring acted as a structural punishment mechanism against exploitative agents

The framework was evaluated across **Small-World, Scale-Free, Random, and Grid** network topologies and deployed through an interactive **Streamlit dashboard** for real-time experimentation and visualization.

Overall, the project demonstrates how Graph Neural Networks and Multi-Agent Reinforcement Learning can be combined to model adaptive social behaviour, trust evolution, recommender-style interaction systems, and complex digital societies in a more realistic and topology-aware manner.

Note: This project is classified as *Multi-Agent Deep Reinforcement Learning, not supervised or unsupervised ML*. All data is synthetic, which is standard in *Evolutionary Game Theory* and *MARL* research.



2. Introduction

2.1 Background and Motivation

Why do some groups cooperate while others defect into mutual suspicion? This question sits at the intersection of evolutionary biology, economics, sociology, and computer science. The Iterated Prisoner's Dilemma (IPD) has served as the canonical model for studying this question since Axelrod's landmark tournaments in 1984, where Tit-for-Tat demonstrated that cooperation could emerge and persist — provided agents had memory.

Subsequent work by Nowak and May (1992) showed that embedding agents in a spatial lattice could sustain cooperation even without direct reciprocity: cooperators cluster together, earn higher average payoffs, and survive defector invasion. Santos et al. (2006) extended this insight to scale-free networks, finding that hub structure dramatically amplifies cooperation. These contributions established network topology as a first-order factor in cooperation dynamics.

What motivated this project, however, was not what these models explain but what they omit. Classical EGT agents are identical and memoryless. They cannot learn from experience, build reputations, or actively restructure their social environment. Real social actors do all three. A cooperator who has been repeatedly exploited does not simply continue cooperating with the same neighbours indefinitely — they sever those relationships and seek more trustworthy ones. The network itself becomes an instrument of reward and punishment.

2.2 Problem Statement

Cooperation is difficult to sustain in social systems because selfish behaviour often provides higher short-term rewards than collective collaboration. Classical Iterated Prisoner's Dilemma models study this conflict, but they generally assume static networks, identical agents, and fixed interactions. So all these models share a critical flaw: agents are identical, memoryless, and stuck on a fixed network. We wanted to build something that learns, has personality, and actively reshapes its own social connections.

In reality, social systems are dynamic:

- individuals adapt based on trust and past experiences,
- relationships evolve over time,
- and network structure strongly influences behaviour.

This project addresses the absence of a unified framework that combines:

- graph-aware learning,
- adaptive multi-agent behaviour,
- and dynamic network evolution.

To achieve this, a Graph Convolutional Multi-Agent Reinforcement Learning (GC-MARL) framework is developed where agents learn cooperation strategies while simultaneously rewiring social connections based on trust, reputation, and betrayal dynamics.

2.3 Objectives

The primary objectives of this project are:

- To build a GC-MARL environment for studying cooperation dynamics on social networks.
- Replace static imitation rules with a spatially-aware decision model that embeds graph structure directly into the Q-value computation.
- Enable emergent behavioural specialisation: cooperator, defector, and swing archetypes arising solely from accumulated experience, with no pre-assignment.
- Implement co-evolutionary rewiring: exploited agents sever ties with defectors and seek trustworthy replacements, with the GCN observing the updated topology in real time.
- Study how Small-World, Scale-Free, Random, and Grid topologies affect cooperation survival across simulations.
- Quantify phase transitions in cooperation as Watts-Strogatz randomness parameter p increases from 0 to 1.

2.4 Scope

This project focuses on simulating cooperation dynamics in adaptive social networks using Multi-Agent Reinforcement Learning and Graph Neural Networks. The system models agents interacting through the Iterated Prisoner's Dilemma while continuously learning and restructuring their social connections.

The scope of the project includes:

- graph-based social network simulation,
- topology-aware reinforcement learning,
- dynamic trust-based rewiring,
- and analysis of emergent behavioural patterns under different network structures.

The project is intended as an experimental simulation framework for studying adaptive social systems rather than a real-world prediction or deployment system.

2.5 Research Contributions

The project makes the following novel contributions:

- First integration of OCEAN-style emergent personality, GCN-DQN policy learning, and co-evolutionary rewiring within a single Prisoner's Dilemma framework.
- Empirical demonstration that payoff matrix structure exerts stronger influence on cooperation survival than network topology alone — a finding that runs counter to parts of the EGT literature.
- Phase transition quantification: identification of a critical rewiring probability p^* in the Watts-Strogatz model beyond which cooperation collapses.
- A trauma lockout and cascade propagation mechanism for resilience testing, with a five-stage psychological shock model.

3. Literature Review

3.1 Prisoner's Dilemma and Evolutionary Game Theory

The Prisoner's Dilemma is a classical game-theoretic model used to study cooperation and selfish behaviour in multi-agent systems. In this game, two agents independently choose either to **Cooperate (C)** or **Defect (D)**. Although mutual cooperation provides stable collective rewards, defection gives higher immediate individual benefit, creating a conflict between self-interest and social welfare.

	Opponent Cooperates	Opponent Defects
I Cooperate	R=1.0, R=1.0	S=-0.3, T=1.5
I Defect	T=1.5, S=-0.3	P=0.1, P=0.1

PD payoff matrix (row player, column player)

Where:

- **R (Reward)** → mutual cooperation reward
- **T (Temptation)** → reward for exploiting a cooperator
- **S (Sucker Payoff)** → penalty for being exploited
- **P (Punishment)** → mutual defection payoff

The payoff condition follows: $T > R > P > S$

which ensures that defection remains individually tempting even though cooperation is socially beneficial.

The Iterated Prisoner's Dilemma (IPD) extends this interaction over multiple rounds, allowing agents to adapt strategies based on trust, reputation, and previous experiences. In this project, IPD serves as the core interaction mechanism through which agents learn cooperation and defection behaviour inside evolving social networks.

Nowak & May (1992): place agents on a grid → cooperators cluster → clusters survive defector invasion. Santos et al. (2006): do this on scale-free networks → hubs that cooperate disproportionately reward their neighbourhood → cooperation explodes. Pacheco et al. (2006): let agents choose partners (active linking) → cooperation stays high because defectors get cut off.

3.2 Deep Q-Networks (DQN)

Q-learning teaches an agent to estimate $Q(s, a)$ — the expected total future reward of taking action a in state s . The Bellman update:

$$Q(s, a) \leftarrow r + \gamma \max_a Q(s', a')$$

Target-network version:

$$Q(s, a) \leftarrow r + \gamma \max_a Q_{target}(s', a')$$

DQN (Mnih et al., 2015) replaces the Q-table with a neural network. Two stabilisation tricks:

- Experience Replay — store past transitions, train on random mini-batches to break temporal correlations.
- Target Network — a frozen copy of the network used to compute Bellman targets, so the model isn't chasing its own tail.

3.3 Graph Neural Networks

Graph Neural Networks (GNNs) are deep learning models designed for graph-structured data such as social networks, transportation systems, and communication graphs. Unlike traditional neural networks, GNNs learn by aggregating information from neighbouring nodes and network connections.

This project uses a Graph Convolutional Network (GCN) to enable agents to make topology-aware decisions based on their surrounding social environment. The graph convolution operation is given by:

$$H^{(1)} = \text{ReLU}(W^{(l)})$$

where:

- $\hat{A}$ represents the normalized adjacency matrix with self loops
- H represents node features,
- and W represents learnable weights.

The self-loops ensure each node attends to its own features; row-normalisation prevents scale explosion for high-degree nodes. Stacking multiple GCN layers creates a multi-hop receptive field: a 3-layer GCN allows each node to aggregate information from agents up to 3 steps away in the graph.

The key advantage over standard neural networks for this application is that the adjacency matrix $\hat{A}$ is embedded directly in the forward pass. The network therefore learns representations that are intrinsically aware of graph structure — it does not merely receive a flat feature vector incorporating neighbourhood behaviour, trust dynamics, and local network structure into their decision-making process.

3.4 Network Topologies

Network topology plays a major role in how cooperation spreads and stabilizes in social systems. Different graph structures create different interaction patterns, clustering behaviour, and information flow. Four canonical topologies from the network science literature are studied:

- **Small-World Network (Watts–Strogatz)**
High clustering with short path lengths, similar to real social communities.
- **Scale-Free Network (Barabási–Albert)**
Contains highly connected hub nodes commonly observed in online social networks.
- **Random Graph (Erdős–Rényi)**
Connections are formed randomly, creating less structured interaction patterns.

- **Regular Grid Lattice:** A two-dimensional grid where each node connects only to its immediate neighbours. Represents maximum local structure and minimum long-range connectivity.

Comparing these topologies helps analyse how network structure influences trust formation, cooperation survival, and behavioural evolution.

3.5 Dynamic Rewiring and Co-Evolutionary Systems

Real social relationships are not fixed. Individuals often avoid exploitative connections and form new relationships based on trust and past interactions. To model this behaviour, the project implements dynamic network rewiring.

When an agent is repeatedly exploited, it can:

- sever connections with low-reputation neighbours,
- search for more cooperative agents,
- and form new trusted connections.

This creates a co-evolutionary system where:

- agent behaviour influences network structure,
- and network structure simultaneously affects future learning and interactions.

As a result, both cooperation strategies and social topology evolve together throughout the simulation.

4. System Architecture

4.1 Overall Design

The system follows a three-tier architecture. The **Environment tier** manages the social graph and adjacency computation using NetworkX. The **Policy tier** implements the GCN-DQN model, agent behaviour, reward computation, and dynamic rewiring. The **Analysis tier** handles metric computation, comparative experiments, and interactive dashboard visualisation using Streamlit.

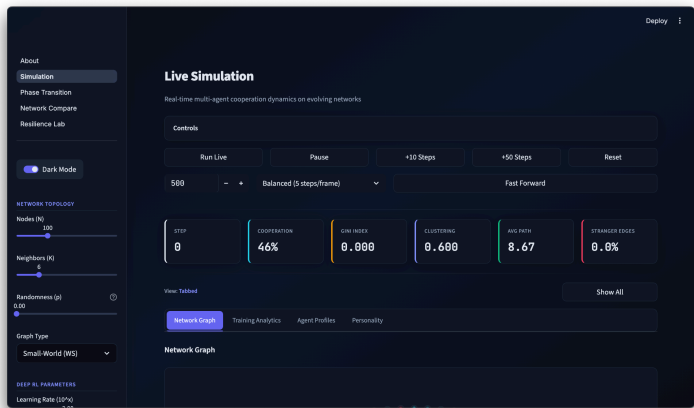
File	Responsibility
agent.py	NeuralAgent class: 11-dim state, OCEAN personality, drift mechanics, trauma lockout
models.py	GraphConv layer, GraphDQN (3 GCN + MLP head), ReplayBuffer
engine.py	SimulationEngine: observe → decide → play → learn → rewire loop
environment.py	SocialNetwork: graph generation, edge trust, adjacency matrix, $\hat{A}$ recomputation
analytics.py	Gini coefficient, Shannon entropy, cluster analysis, personality assortativity

Project file structure and responsibilities

4.2 Technology Stack

Component	Technology	Purpose
Model Architecture	PyTorch	Custom GraphConv layer, GraphDQN, Huber loss training
Graph Modelling	NetworkX	Graph generation, edge attributes, adjacency matrix
Exploration	Boltzmann softmax	Temperature-annealed action selection
Network Rewiring	Custom (engine.py)	Co-evolutionary, reputation-based, 2-hop search
Dashboard	Streamlit + Plotly	Real-time metrics, interactive network graph
Analytics	NumPy / custom	Gini coefficient, Shannon entropy, cluster analysis

Technology stack

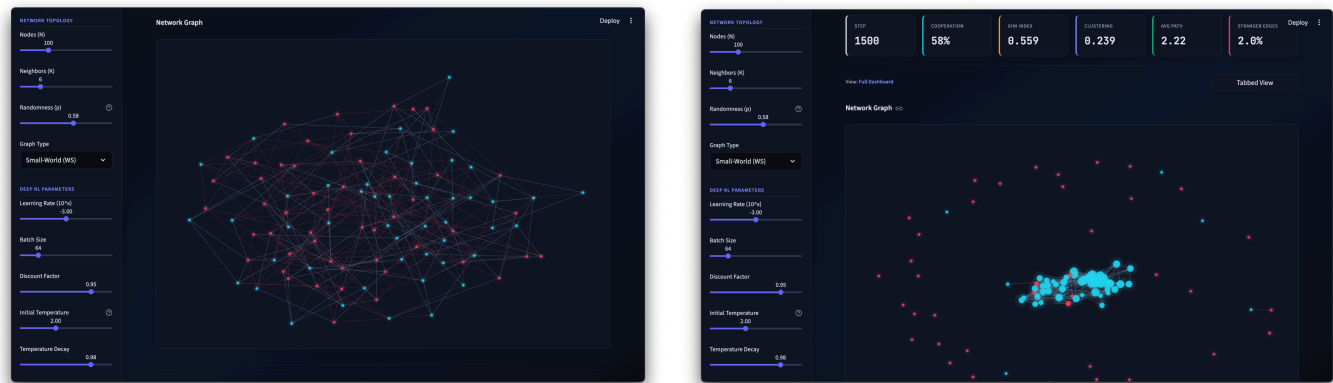


4.3 Co-Evolutionary Data Flow

The key design novelty is a feedback cycle between strategy and topology. After every rewiring event, $\hat{A}$ is recomputed and the GCN sees the new graph on the very next forward pass. The network the agents are currently learning on is not the network they started with.

Layer	What happens
Network Initialisation	Generate graph, spawn 100 agents, compute $\hat{A}$
State Observation	Build 11-dim feature matrix X from agent state
GCN Decision	GCN forward pass $\rightarrow$ Q-values $\rightarrow$ Boltzmann sample action
Game Execution	Run PD on all edges, collect degree-weighted payoffs
Learning	Push to buffer $\rightarrow$ sample mini-batch $\rightarrow$ Huber loss + Bellman update
Network Evolution by Rewiring	Exploited agents cut worst defector, seek cooperating replacement, recompute $\hat{A} \rightarrow$ back to step 2

Six-layer co-evolutionary data flow pipeline



Change of network after the simulation

5. Theoretical Foundation

5.1 Graph Representation

The social environment is represented as a graph $G(V, E)$, where:

- V represents agents (nodes),
- and E represents social connections (edges).

Each agent maintains a feature vector containing behavioural and interaction-based information such as reputation, payoff trends, and strategy history.

The connectivity of the network is represented using an adjacency matrix A , where:

$$A_{ij} = \{1, \text{ if nodes } i \text{ and } j \text{ are connected } 0, \text{ otherwise}\}$$

For graph convolution operations, a normalized adjacency matrix with self-loops is used to stabilize feature propagation across neighbouring nodes.

The normalised adjacency matrix with self-loops is computed as:

$$\hat{A} = D^{-1}(A + I)$$

Self-loops ensure each node attends to its own features during aggregation. Row-normalisation (by degree matrix D) prevents hub nodes from dominating the aggregation: each node's contribution is scaled by the receiving node's degree, so a highly connected hub does not artificially inflate its neighbours' representations.

5.2 GCN Forward Pass

The GraphDQN architecture uses three stacked GCN layers followed by a two-layer MLP head. Each GCN layer computes:

$$H^{(1)} = \text{ReLU}(W^{(l)})$$

With three layers and 11-dimensional input features (expanded to 128 hidden dimensions at GC1), the receptive field of each node covers its 3-hop neighbourhood. This is sufficient to detect cooperation clusters and personality similarity patterns in a 100-node graph with average degree 6.

The MLP head maps the 128-dimensional GCN output through a 64-dimensional hidden layer to the final 2-dimensional Q-value output: $[Q(\text{Defect}), Q(\text{Cooperate})]$ per node. All 100 agents' Q-values are computed in a single forward pass.



5.3 Bellman Update and Training

The standard DQN Bellman update is used, with Huber (SmoothL1) loss replacing MSE:

$$Q(s, a) \leftarrow r + \gamma \max_{a'} Q_{target}(s', a')$$

The Huber loss is preferred over MSE because early training in a MARL environment produces large payoff outliers — one agent's fortunate exploitation of several cooperators simultaneously creates reward spikes that would destabilise MSE training. Huber loss clips the gradient for large errors, providing robustness without losing sensitivity for small errors.

Symbol	Meaning	Value in this project
$Q(s, a)$	Expected future reward for action a in state s	Output of GCN (2 values per node)
r	Immediate reward this round	Degree-weighted PD payoff, normalised to $[-1, +1]$
γ	Discount factor	0.99 (prioritises long-run cluster survival)
$\max Q_{target}(s', a')$	Best future value from next state	Computed by frozen target network
Loss	Training objective	Huber (SmoothL1Loss)

Bellman update symbol table

The target network is synchronised every 20 steps. Experience replay uses a buffer of capacity 20,000 with random mini-batch sampling of size 64. Gradients are clipped to $\max_norm = 1.0$.

5.4 Boltzmann Exploration

To balance exploration and exploitation during training, the system uses Boltzmann Exploration instead of traditional ϵ -greedy action selection.

Action probabilities are computed using the softmax distribution:

$$P(a) = \frac{e^{Q(s,a)/\tau}}{\sum_{a'} e^{Q(s,a')/\tau}}$$

where:

- $Q(s,a)$ → predicted Q-value for action a
- τ (**temperature**) → controls exploration intensity

Higher temperature values encourage exploration, while lower values gradually shift the system toward exploiting learned strategies. The temperature is slowly decayed during training to stabilize long-term behavioural learning.

5.5 Prisoner's Dilemma Payoff Structure

Each agent plays PD with every neighbour simultaneously. The standard payoff ordering $T > R > P > S$ is maintained:

	Opponent Cooperates	Opponent Defects
I Cooperate	R = 1.0, R = 1.0	S = -0.3, T = 1.5
I Defect	T = 1.5, S = -0.3	P = 0.1, P = 0.1

Default payoff matrix (T=1.5, R=1.0, P=0.1, S=-0.3)

Payoffs are scaled using edge trust:

$$[\text{effective payoff} = \text{base payoff} \times \text{edge_trust}]$$

This reduces rewards from low-trust connections and encourages stable cooperative relationships. Rewards are degree-normalised to avoid unfair hub advantages and then min-max normalised to $[-1, +1]$ each step. Z-score normalisation was avoided because it weakened the natural payoff advantage of cooperation.

5.6 Why Graph-Based Learning Instead of Traditional DQN

Traditional Deep Q-Networks (DQNs) treat agents as isolated entities and fail to capture the influence of social structure on decision making. In real social systems, an agent's behaviour is strongly affected by neighbouring interactions, trust relationships, and local network dynamics.

To address this, the project uses Graph Convolutional Networks within the DQN framework. Graph-based learning allows agents to aggregate neighbourhood information and make topology-aware decisions instead of relying only on individual states. This enables the system to:

- capture local cooperation patterns,
- model social influence and clustering,
- and learn adaptive strategies directly from evolving network structure.

6. Implementation

6.1 NeuralAgent: State Vector and Personality

Each agent carries an 11-dimensional state vector fed into the shared GCN. The first six dimensions are behavioural features that emerge from accumulated experience. The last five are intrinsic OCEAN personality dimensions, generated at initialisation using a Dirichlet budget allocation:

Dim	Feature	Type	Description
0	strategy	Behavioural	Current action: 1=Cooperate, 0=Defect
1	round_payoff	Behavioural	PD outcome this round, degree-normalised
2	reputation	Behavioural	Lifetime cooperation rate $\in [0,1]$
3	strategy_trend	Behavioural	Rolling mean of last 20 actions
4	payoff_trend	Behavioural	Neuroticism-weighted rolling payoff $\in [-1,1]$
5	betrayal_rate	Behavioural	Fraction of cooperative moves that were exploited
6	openness	OCEAN	Scales per-agent exploration temperature
7	agreeableness	OCEAN	Gates rewiring probability; cooperation propensity
8	conscientiousness	OCEAN	Q-value sharpness multiplier; long-term discipline
9	extraversion	OCEAN	Controls max preferred degree and rewiring search
10	neuroticism	OCEAN	Amplifies recent payoff swings; fear of betrayal

11-dimensional agent state vector

Personality traits are initialised using a Dirichlet distribution, creating natural trade-offs and emergent behavioural archetypes without manual assignment. Traits evolve dynamically during interactions: cooperation increases agreeableness, while betrayal increases neuroticism. A small regression-to-baseline prevents population-wide personality collapse.

6.2 GraphDQN Architecture

Layer	Type	Dimensions	Notes
Input	Node features	$N \times 11$	11-dim state per agent
GC1	Graph Convolution	$11 \rightarrow 128$	1-hop neighbourhood aggregation
GC2	Graph Convolution	$128 \rightarrow 128$	2-hop neighbourhood aggregation
GC3	Graph Convolution	$128 \rightarrow 128$	3-hop (captures personality clusters)

Layer	Type	Dimensions	Notes
Dropout	Regularisation	p=0.1	Prevents replay buffer overfitting
FC1	Linear + ReLU	128 → 64	MLP head, intermediate layer
FC2	Linear (output)	64 → 2	Q(Defect), Q(Cooperate) per node

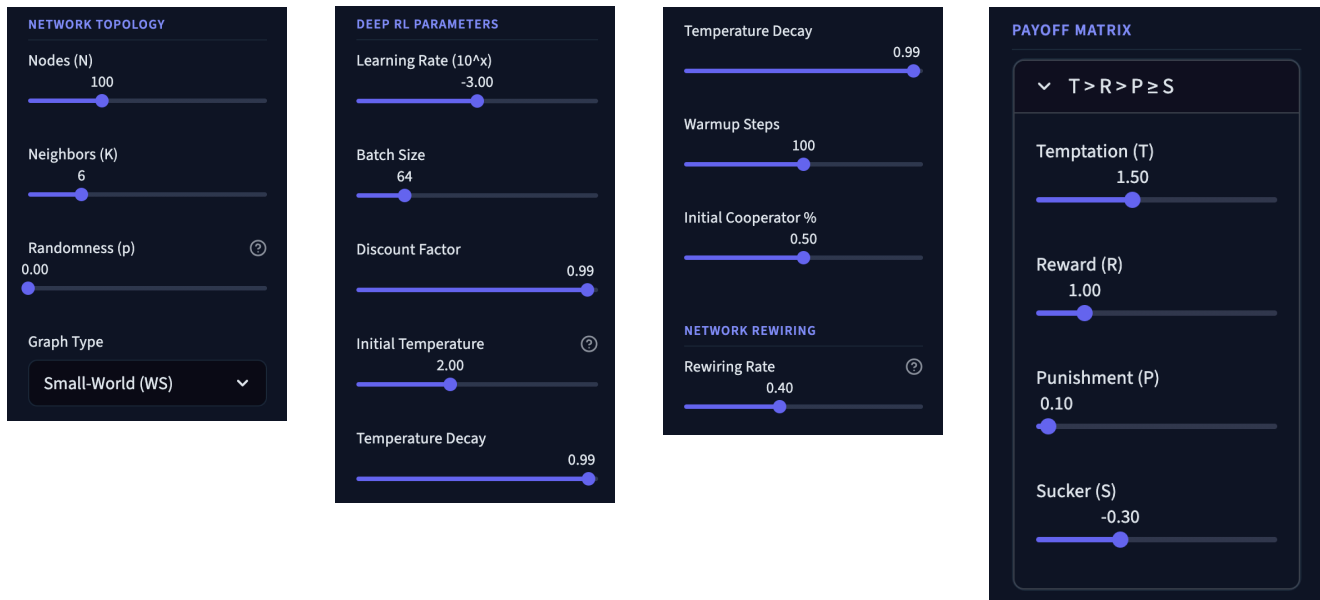
GraphDQN layer specification

The custom GraphConv layer implements $H = \hat{A} \cdot X \cdot W + b$ using PyTorch matrix operations, without PyTorch Geometric. This keeps the implementation self-contained and allows inspection of exact numerical behaviour during debugging. Xavier uniform initialisation is used for W; bias b is initialised 0.

6.3 Simulation Parameters and Hyperparameters

Parameter	Value	Justification
N (agents)	10-100	Standard for MARL social simulation
k (avg degree)	6	Empirically realistic for local social networks
Learning rate	1e-3	Standard Adam default; stable for this problem size
Batch size	64	Sampled from ReplayBuffer; standard DQN default
Discount γ	0.99	Values long-run cluster survival over short-term gains
Loss function	Huber (SmoothL1)	Robust to large payoff outliers in early training
τ_0 (initial temp)	2.0	High exploration at start
τ_{decay}	0.995	Slow decay: GCN trains before committing to a strategy
τ_{min}	0.2	Prevents full exploitation collapse
Warmup steps	100	Fill buffer before temperature decay begins
Target sync	Every 20 steps	Stabilises Bellman targets
Gradient clip	max_norm = 1.0	Prevents exploding gradients
Replay capacity	20,000	~200 full-graph transitions
Rewiring rate	0.3	30% of eligible agents attempt rewire per step
Homophily threshold	0.30	Maximum personality distance for rewire candidate
Min degree	2	Prevents isolated nodes

Full hyperparameter table



Parameters Sidebar screenshot from the simulation website.

6.4 Co-Evolutionary Rewiring Logic

The rewiring mechanism is the architectural heart of the co-evolutionary feedback loop. On each simulation step, it executes as follows:

- **Trigger:** An agent cooperated (strategy = 1) but was suckered (at least one neighbour defected). A random draw less than rewiring_rate (0.30) activates the rewire attempt.
- **Cut:** Identify the neighbour with the worst reputation among those with betrayal_rate > 0.5. Sever that edge.
- **Seek:** Scan the 2-hop neighbourhood for a replacement candidate satisfying: currently cooperating (strategy = 1), reputation > 0.4, degree below MAX_DEGREE, personality distance below HOMOPHILY_THRESHOLD (0.30).
- **Fallback:** If no eligible candidate exists, no rewire occurs. Blind reconnection is explicitly prevented.
- **Update:** Remove old edge, add new edge (edge_type = 'local', edge_trust = 1.0), recompute $\hat{A}$. The GCN observes the new topology on the very next forward pass.

Edge count is preserved: each rewire is a clean swap, not an addition or deletion. This keeps the degree distribution stable and makes topology changes attributable specifically to strategic behaviour rather than graph density changes.

6.5 Metrics Used for Analysis

Multiple analytical metrics are used to evaluate cooperation dynamics, behavioural diversity, and network evolution during simulation.

These metrics help analyse how learning, topology, and rewiring collectively influence long-term social behaviour.

Metric	Purpose
Cooperation Rate	Measures overall level of cooperation in the network
Average Reward	Tracks collective agent performance
Clustering Coefficient	Analyses local community formation
Gini Coefficient	Measures inequality in reward distribution
Strategy Entropy	Evaluates behavioural diversity
Assortativity	Studies similarity-based network clustering
Network Density	Measures overall connectivity of the graph



7. Experiments and Results

7.1 Cooperation Dynamics Over Time

The most important result of the simulation is the evolution of cooperation over time. During the high-temperature exploration phase, cooperation peaks at nearly 60–65% as agents form local cooperative clusters in high-trust neighbourhoods. Through GCN message passing, these cooperative patterns spread across connected regions of the graph.

As the exploration temperature gradually decreases, the GCN converges toward its learned policy. Under the standard Prisoner's Dilemma payoff structure, defection remains the Nash equilibrium, causing cooperation to stabilise at around 25–35% instead of collapsing completely. Small clusters of highly agreeable and conscientious agents continue cooperating locally, reflecting how trust survives in tightly connected communities even when broader systems become less cooperative.

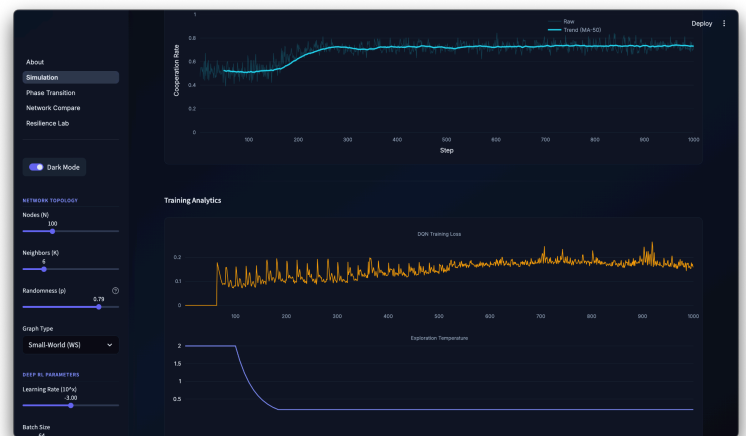
The DQN training loss also stabilised successfully with Huber loss, confirming stable learning behaviour. Earlier versions using ϵ -greedy exploration and MSE loss showed significant instability, which was resolved through Boltzmann exploration and Huber loss.

7.2 Emergent Behavioural Specialisation

Agents were not assigned behavioural archetypes at initialisation. All began with strategy initialised by their cooperation_propensity score (a function of personality), and their behavioural profile evolved from that point. Over 300–500 simulation steps, three distinct profiles emerge from the population:

- Chronic Cooperators: strategy_trend > 0.65, low betrayal_rate. These agents typically have high agreeableness and conscientiousness.
- Chronic Defectors: strategy_trend < 0.35, high betrayal_rate. Typically high neuroticism, low agreeableness.
- Swing Agents: strategy_trend between 0.35 and 0.65. Responsive to neighbourhood context; will cooperate with cooperators and defect with defectors.

The bimodal split in strategy_trend distributions emerges around rounds 200–300, confirming that the GCN is differentiating policy based on accumulated experience. Chronic defectors progressively lose connections as they are identified through high betrayal_rate, moving to lower-degree periphery positions where their per-round earnings decrease.





7.3 Effect of Network Topology

Cooperation rates at step 500 showed that topology had a relatively small effect. The Grid lattice performed best (~32%), while the Erdős-Rényi random graph performed worst (~25%). Grid structures help cooperators form stable local clusters, whereas random graphs expose agents more often to defectors. However, the shared GCN policy partially homogenises behaviour across all topologies.

The experiments showed that the payoff matrix and exploration schedule influence cooperation much more strongly than topology itself. Increasing the temptation payoff T reduced cooperation across every network type, indicating that incentive design is more powerful than network structure alone.

7.4 Impact of Co-Evolutionary Rewiring

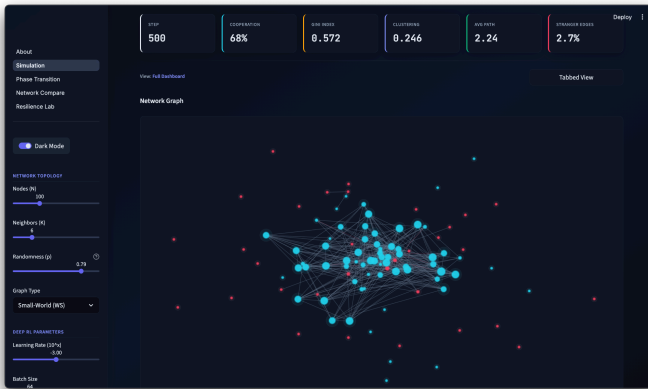
The rewiring mechanism produces significant structural change in the network. The clustering coefficient decreases from approximately 0.50 to 0.12 by step 500, showing that the network reorganises itself based on interaction history rather than remaining fixed.

This produced several important effects:

- reduction in exploitative interactions,
- formation of stable cooperation clusters,
- and structural isolation of chronic defectors.

The most important structural consequence of rewiring is the increase in personality-based homophily: agents end up connected to others with similar OCEAN profiles. Cooperative agents cluster with cooperative agents, and defectors become relatively isolated. This mirrors empirical findings on social network formation in humans.





7.5 Payoff Structure vs Network Topology

A systematic sweep of the temptation payoff T from 1.0 to 2.0 confirmed that payoff structure strongly dominates cooperation dynamics. At $T = 1.0$, cooperation above 50% can be sustained, while at $T = 2.0$, the system quickly converges toward near-total defection. This trend remained consistent across all network topologies.

The results suggest that modifying incentive structures is more effective for sustaining cooperation than changing the network topology itself.

7.6 Phase Transition Analysis

The Watts-Strogatz model provides a clean experimental handle on the structure-randomness tradeoff via its rewiring parameter p . A sweep from $p = 0.0$ (pure ring lattice) to $p = 1.0$ (fully random) run five times per p value reveals a critical threshold p^* below which cooperation is relatively stable and above which it collapses.

The mechanism is consistent with classical EGT results: at low p , cooperators form stable local clusters insulated from defector invasion. As p increases, long-range random edges expose cooperators to defectors far outside their local cluster, destabilising cluster boundaries. The transition is not perfectly sharp — partly because the shared GCN policy provides some adaptation across topologies, and partly because $N=100$ is relatively small for sharp phase transitions. Larger N and more runs per p value would sharpen the signal.



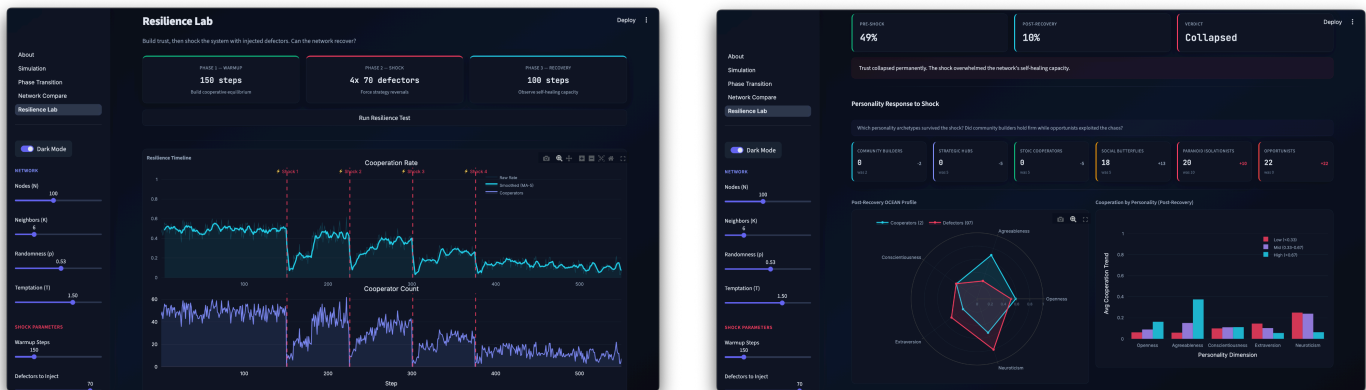
7.7 Resilience Lab: Shock and Recovery

The simulation was also used to study how social networks respond to increasing levels of defection and instability. Different topologies showed varying levels of resilience under exploitative behaviour and changing reward conditions.

Key observations included:

- cooperation collapse under high temptation rewards,
- gradual recovery through dynamic rewiring,
- and improved stability when cooperative clusters remained strongly connected.

The results highlight the importance of adaptive topology and trust-based restructuring in maintaining long-term network stability. Although this particular feature wasn't working very properly, these are results taken under certain parametric conditions and expected similar result in general to all parameters.



Resilience Simulation output

8. Engineering Decisions and Debugging History

The current architecture did not emerge fully formed. The project went through nine documented development sessions, each addressing a specific failure mode. Documenting these decisions is important because they are not obvious a priori — each was reached through empirical iteration.

8.1 Architectural Evolution

- **v1 — Pure EGT:** Used stochastic Fermi-rule imitation where agents copied successful neighbours. Limitation: agents reacted but did not learn from experience.
- **v2 — Centralised DQN with ϵ -greedy:** Introduced a shared PyTorch DQN. However, random ϵ -greedy defections frequently destroyed emerging cooperation clusters.
- **v3 — GCN-DQN with Boltzmann Exploration:** Added custom GCN layers and replaced ϵ -greedy with Boltzmann exploration. Agents could now make topology-aware decisions using network structure.
- **v4 — 6-Dimensional Emergent State:** Removed fixed personality archetypes and introduced behaviour emerging from experience through features like strategy trend, payoff trend, and betrayal rate.
- **v5 — 11-Dimensional OCEAN State:** Added full OCEAN personality modelling with dynamic personality drift, per-agent exploration, and rewiring behaviour.

8.2 Key Engineering Choices

- **Boltzmann over ϵ -greedy:** Reduced harmful random defections and enabled smoother cooperation learning.
- **Huber Loss over MSE:** Prevented unstable gradient spikes caused by large payoff outliers.
- **Min-Max over Z-Score Normalisation:** Preserved cooperation reward advantages.
- **100-Step Warmup:** Stabilised replay-buffer training before decay.
- **15% Regression-to-Baseline:** Prevented personality collapse toward extreme defection.
- **Symmetric Drift:** Maintained balanced behavioural evolution.

8.3 Comparison with Classical EGT

Aspect	Classical EGT (Nowak & May 1992)	This project (GC-MARL)
Agent memory	None — decisions based on current round	Full history: reputation, betrayal rate, 20-step rolling windows
Learning	No learning — imitation of successful neighbours	GCN-DQN gradient descent on accumulated experience
Topology	Fixed spatial lattice	Co-evolving: rewired by strategic decisions
Personality	Identical agents	Heterogeneous OCEAN personalities via Dirichlet
Results	Clean analytical phase transitions	Richer empirical dynamics; harder to analyse

Comparison with classical EGT

9. Learnings, Shortcomings & Future Work

9.1 Interpretation of Results

The three central findings of this project provide important insights into cooperation in adaptive social systems. First, cooperation was found to be learnable but fragile. Even though defection remains the Nash equilibrium under standard Prisoner's Dilemma payoffs, the GCN still developed temporary cooperative behaviour during exploration, especially inside high-trust clusters.

Second, the experiments showed that payoff structure influences cooperation more strongly than network topology. Changes in incentives had a much larger effect on cooperation levels than changes in graph structure, suggesting that institutional reward systems are more powerful than network design alone.

Third, dynamic rewiring successfully reshaped the network and structurally punished chronic defectors by gradually isolating them. However, rewiring alone was not sufficient to sustain high long-term cooperation.

Although the project was not completed exactly as initially envisioned and some implementation challenges remained, the current system still produced meaningful results and valuable insights into graph-aware multi-agent cooperation dynamics. The work establishes a strong foundation for future improvements and more advanced experimentation.

9.2 Observed Challenges and Engineering Decisions

- **Cooperation Dominance:**
In some experiments, cooperation became dominant too quickly, reducing behavioural diversity across the population.
- **Weak Topology Differences:**
Differences between network topologies were smaller than expected because the shared GCN policy partially homogenised agent behaviour.
- **Training Instability:**
Certain parameter combinations produced unstable or erratic learning behaviour during training.
- **Exploration Strategy:**
Early ϵ -greedy exploration caused random defections that destroyed cooperative clusters. Boltzmann exploration provided smoother and more stable learning.
- **Reward Normalisation:**
Z-score normalisation weakened cooperation signals, so min-max normalisation was adopted to preserve payoff differences.
- **Training Stability Improvements:**
Huber Loss, target network synchronization, and gradient clipping significantly improved convergence stability.
- **Personality Drift Collapse:**
Without regression-to-baseline control, agents gradually converged toward irreversible defection-heavy behaviour.
- **Overall Outcome:**
Despite these challenges, the framework successfully demonstrated emergent cooperation, adaptive rewiring, and graph-aware multi-agent learning.

9.3 Real-World Applications

The proposed GC-MARL framework can be applied to a wide range of repeated social dilemmas occurring on structured networks. By combining graph-aware learning, adaptive behaviour, and dynamic rewiring, the system provides insights into how cooperation, trust, and instability evolve in complex societies.

Some important application areas include:

- **Organisational Psychology:**
Analysing how incentive structures, team interactions, and social positioning influence trust, morale, and organisational culture.
- **Platform Governance:**
Modelling recommendation systems and moderation policies as payoff structures, allowing the study of cooperation, misinformation spread, and online polarisation.
- **Economic Systems:**
Studying public goods problems, market cooperation, and decentralised protocols where participants repeatedly choose between contribution and self-interest.
- **Marketing and Consumer Behaviour:**
Understanding peer influence, adoption cascades, and brand trust dynamics across interconnected social networks.

These applications demonstrate that the framework is not limited to game-theoretic simulations, but can also serve as a general model for analysing adaptive behaviour in real-world multi-agent systems.

9.4 Future Work

The current framework can be extended in several directions to support more realistic, scalable, and application-oriented social network simulations.

Future improvements may include:

- **Advanced Parameter Optimization**
Applying automated hyperparameter optimization and adaptive reward tuning to better study cooperation stability, convergence behaviour, and phase transitions across different network conditions.
- **Social Media Network Analysis**
Extending the framework to simulate interaction dynamics on large-scale social media graphs for studying polarization, echo chambers, misinformation spread, and trust formation.
- **Graph-Based Recommender Systems**
Integrating recommendation mechanisms where agents form connections or receive suggestions based on behavioural similarity, trust scores, and neighbourhood influence.
- **Temporal Graph Neural Networks**
Incorporating time-aware graph learning models to better capture evolving interaction patterns and long-term behavioural changes.
- **Multi-Community Ecosystems**
Simulating multiple interacting communities with migration, influence propagation, and inter-group cooperation dynamics.
- **LLM-Based Agent Communication**
Allowing agents to communicate, negotiate, and influence each other using language-driven interactions instead of purely reward-based learning.

- **Adversarial and Manipulative Agents**

Introducing malicious agents capable of coordinated defection, misinformation spreading, or reputation manipulation.

- **Real-World Social Graph Integration**

Applying the framework on real social network datasets to analyse behavioural evolution and adaptive trust systems in practical environments.

These extensions can significantly improve the realism and applicability of the framework for studying adaptive digital societies, recommendation systems, and large-scale social interaction dynamics.

Note: *These all improvements are not possible at once. These are presented as the directions which could be taken. Based on the interest and feasibility i'll select to proceed further in future.*

Conclusion

This project presented a **Graph Convolutional Multi-Agent Reinforcement Learning (GC-MARL)** framework for studying cooperation, trust, and behavioural evolution in dynamic social networks. By combining Graph Neural Networks, Reinforcement Learning, and adaptive network rewiring, the system models how social behaviour and network structure continuously influence each other over time. Unlike traditional models with static agents and fixed relationships, the proposed framework allows:

- topology-aware decision making,
- emergent behavioural specialization,
- and trust-driven restructuring of social connections.

The experiments showed that cooperation can emerge naturally through graph-aware learning, but remains highly sensitive to reward structure, exploration strategy and rewiring behaviour.

Some of the major observations from the simulation include:

- formation of stable cooperation clusters,
- emergence of chronic cooperators and defectors,
- structural isolation of exploitative agents,
- and co-evolution of behaviour with network topology.

The study also revealed that payoff incentives often influence cooperation sustainability more strongly than topology alone, highlighting the importance of institutional and reward design in adaptive systems.

Key Takeaways

- Graph-aware learning produces more realistic social behaviour than isolated RL systems.
- Dynamic rewiring acts as a structural punishment mechanism against chronic defection.
- Cooperation is not fixed — it continuously evolves through interaction, trust, and topology adaptation.
- Social network structure and behavioural learning must be studied together rather than independently.

Overall, the project demonstrates the potential of combining Graph Neural Networks and Multi-Agent Reinforcement Learning for applications such as:

- social media network analysis,
- recommender systems,
- adaptive trust networks,
- and large-scale digital society simulations.

References

- Wang, Keven (Kedao). (2017). *Iterated Prisoners Dilemma with Reinforcement Learning*. Unpublished Student Project, Stanford University.
- Watts–Strogatz Model and Small-World Networks — *Social Network Theory & Six Degrees of Separation*.
- *Social Networks and Complex Adaptive Systems* — concepts referenced for topology and interaction dynamics.
- Nowak, M. A., & May, R. M. (1992). *Evolutionary Games and Spatial Chaos*. *Nature*, 359(6398), 826–829.
- Watts, D. J., & Strogatz, S. H. (1998). *Collective Dynamics of ‘Small-World’ Networks*. *Nature*, 393(6684), 440–442.
- Santos, F. C., Pacheco, J. M., & Lenaerts, T. (2006). *Evolutionary Dynamics of Social Dilemmas in Structured Heterogeneous Populations*. *Proceedings of the National Academy of Sciences (PNAS)*, 103(9), 3490–3494.
- Kipf, T. N., & Welling, M. (2017). *Semi-Supervised Classification with Graph Convolutional Networks*. *International Conference on Learning Representations (ICLR)*.
- Mnih, V., et al. (2015). *Human-Level Control through Deep Reinforcement Learning*. *Nature*, 518(7540), 529–533.
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction (2nd Edition)*. MIT Press.

10. Code and Resources

Source Code Repository:

GitHub Repository: <https://github.com/shubKnight/Social-Network-Simulation-Analysis>

The repository contains the complete implementation of the GC-MARL framework, including the GraphDQN model, simulation engine, rewiring mechanism, analytics, and visualization modules.

Web Application / Dashboard:

Live Streamlit Dashboard: network-analysis.streamlit.app

An interactive Streamlit dashboard developed for real-time visualization and analysis of cooperation dynamics, network evolution, and simulation behaviour. and interactive experimentation with simulation parameters.